

Stage 4 Project Report

2026-04-28

SceneTrip: Movie-Based Trip Planner

Link to the Project Video: https://mediaspace.illinois.edu/media/t/1_2ajlku49

1. Changes in Project Direction

Our final project continues in the same direction as our original proposal: building a movie-based travel planner that creates an itinerary based on a user's favorite movie. SceneTrip allows users to explore filming locations from movies they enjoy and provides airport suggestions, flight route information, estimated travel time, and an estimated trip budget.

In terms of the main application idea, very little changed throughout the project. We still focused on connecting movies, filming locations, nearby airports, and flight routes. The core purpose of the project remained the same from the proposal to the final version.

However, our final project changed slightly because we shifted away from feature completeness and focused more on creating a realistic working demo. In our original proposal, we planned to recommend books related to the input movie. After starting development, we decided to remove this extra feature because it would make the program overly complicated and less focused.

Connecting books to movies would require a new dataset, a separate schema design, and additional recommendation logic that does not directly support the main goal of generating a travel itinerary. By removing this feature, we were able to spend more time polishing the movie-to-itinerary workflow instead of spreading our work across multiple recommendation systems.

In the final version, the database schema, tables, backend logic, and frontend integration were completed around the main travel-planning workflow. We made sure that the different parts of

the project connected together in the frontend so users could enter movie information and receive useful information about a possible trip.

2. Usefulness of the Application

We consider our application useful because it helps users move from movie inspiration to a basic travel plan. Users can enter a movie they like and see how they could travel to related filming locations using airport data, flight routes, and estimated costs.

SceneTrip meets the usefulness goal because it connects information that users would usually need to search for separately. Instead of manually looking up filming locations, nearby airports, possible flights, and rough costs, users can see these pieces of information together in one application.

At the same time, our application does not include every possible travel-planning feature. For example, the flight costs are estimated rather than based on real-time flight prices for a specific travel date. Also, the final version focuses on planning a trip based on one movie instead of combining multiple movies into one itinerary. These would be good areas for future improvement.

3. Data Source or Schema Changes

We ended up using the same general data sources that we outlined in our proposal. Movie data, airport and location data, and flight route data were still necessary because our application depends on connecting movies to filming locations, nearby airports, and possible flights.

The schema changed throughout the project as we moved from a high-level proposal to a real implementation. In the final version, we used tables such as users, movies, actors, locations, flights, trip_plans, trip_plan_stops, booked_flights, movie_actors, and movie_locations.

This final structure made it easier to support real database queries, many-to-many relationships, ordered itinerary stops, and generated flight legs. It also helped us separate static dataset information, such as movies and locations, from user-generated information, such as saved trip plans.

4. ER Diagram and Table Design Changes

We kept the same main concepts from our original ER diagram. The project still includes users, movies, actors, locations, flights, and trip plans. The main idea is that a user chooses a movie and the application connects that movie to filming locations and travel information.

The biggest change was moving from a general ER diagram to relational tables that could support a working database application. Instead of storing repeated information or comma-separated values in one table, we used relational design to organize the data more clearly.

For example, we used bridge tables such as `movie_actors` and `movie_locations` to handle many-to-many relationships. We also added tables such as `trip_plan_stops` and `booked_flights` to store ordered itinerary stops and flight legs. This design reduces redundancy, improves query performance, and makes the database easier to maintain.

5. Functionalities Added or Removed

One functionality we added in the final application is a complete login flow with demo users.

This allows generated trip plans to be associated with a user account.

We also added a clear search form where users can enter a movie title, departure city, and budget.

After submitting the form, users are shown a results page with filming stops, flight information, total travel days, and a budget breakdown.

Some planned features were simplified or removed. Originally, we wanted to build a more complete trip-planning solution, including stronger route optimization and multi-movie trip planning. In the final version, we focused on generating a usable itinerary from one favorite movie. This was a better choice because it allowed us to finish the full application flow and make sure the frontend, backend, and database worked together correctly.

6. How Advanced Database Programs Fit into the Application

Transactions are used when loading generated trips into the database. Creating a trip involves multiple inserts and relationships, including adding a trip plan, ordered stops, and related flight legs. Transactions make sure that either all of these steps are completed or none of them are. For example, if a valid flight connection cannot be created, the operation can be rolled back so the database does not store an incomplete trip plan.

Stored procedures help organize the query logic for generating trips. Finding flights between locations while considering budget and travel days requires multiple query steps. By placing this logic inside stored procedures, we keep the application code cleaner and make the trip-generation logic easier to reuse and maintain.

Triggers and constraints help enforce data integrity. Foreign key constraints make sure that records in trip plans, locations, and flights reference valid data. These constraints prevent invalid relationships from being inserted into the database. Triggers could also be used for additional validation, although much of our insert logic was already handled in the application layer.

Overall, these advanced database programs make the application more reliable. Transactions help prevent partial data, stored procedures move complex logic into the database layer, and

constraints help keep the database consistent. Together, they make SceneTrip more practical than a simple data display project.

7. Technical Challenges

Team Member 1: Jim Xiao

One technical challenge faced by him was integrating multiple datasets. The movie dataset, airport dataset, and flight route dataset did not share a clean common key. For example, filming locations had to be connected to real cities and airport codes, but the naming formats were not always consistent. A future team working on a similar project should spend enough time cleaning and standardizing names before building the application logic.

Team Member 2: Yilin Wang

He faced a technical challenge while designing the database schema. The project includes many relationships, such as movies to actors, movies to locations, and trip plans to stops and flights. If these relationships are not separated properly, the database can become redundant and difficult to query. A useful lesson is to use bridge tables for many-to-many relationships and to separate user-generated data from static dataset records.

Team Member 3: Peiyuan Sun

One technical challenge he faced was route feasibility. Some filming locations do not have direct flights from the user's departure city or between each other. Without extra logic, the system could create unrealistic routes. We decided to display a warning when a route requires transfers or when users may need to search for flight connections manually. A future team should address this issue early and design the interface to clearly explain route limitations.

Team Member 4: Ze Chen

He faced a technical challenge when connecting the frontend with the backend and database.

Each movie search requires several backend steps, including finding the movie, retrieving filming locations, finding nearby airports, checking flights, and estimating the budget. A future team should test individual API endpoints before combining them into the full frontend search workflow.

8. Other Changes Compared with the Original Proposal

Compared with the original proposal, the final project became more focused on the main user workflow. Instead of trying to build every feature a user might want in a travel planner, we focused on demonstrating a working product.

The final version emphasizes logging in, searching for a movie, viewing filming locations, seeing route explanations, and receiving a budget estimate. Overall, the project did not change direction. It became more practical by narrowing the scope and making sure the core features worked together.

9. Future Work

One clear way to improve SceneTrip is to improve the frontend interface. Users should be able to see how routes were generated and understand when some flight connections must be searched manually.

There are several features we could add in the future. First, we could use a real-time flight API so the budget estimate is based on current flight prices. Second, we could allow users to enter multiple favorite movies and generate one combined itinerary. Third, we could improve the

recommendation algorithm so it balances movie relevance, distance, flight availability, total travel days, and budget.

We could also add hotel and local transportation estimates to make the total trip budget more realistic. Finally, we could implement a stronger route optimization heuristic or shortest-path algorithm for searching the flight network. This would help the application produce better routes instead of only filtering flights by budget.

10. Final Division of Labor and Teamwork

Our team divided the work based on the main parts of the project.

Team Member 1: Jim Xiao worked on the frontend pages, including the home page, login modal, search form, and results page.

Team Member 2: Ze Chen took responsibility for backend API endpoints and connected the frontend search functionality to database queries.

Team Member 3: Peiyuan Sun is responsible for database schema design, data loading, advanced SQL queries, and indexing.

Team Member 4: Yilin Wang's job is data cleaning, application testing, report writing, and presentation slides.

Overall, our team worked well together by dividing the project into frontend, backend, database, and report/presentation tasks. This allowed us to work in parallel while keeping the final application consistent. When we realized the original scope was too large, we adjusted the project together so we could complete a working application instead of leaving several unfinished features.

Conclusion

SceneTrip follows the same main direction as our original proposal: creating a database-backed movie-based travel planner. Users can log in, search for a movie, enter a departure city and budget, and receive information about a possible trip. The final itinerary includes filming locations, airport suggestions, flight information, route explanations, and an estimated budget.

Although the project scope became narrower than the original proposal, the final version demonstrates a realistic use case with working frontend, backend, and database connections. By focusing on the core movie-to-itinerary workflow, our team was able to build a complete and practical application.